

CPT202

MELODIFY: MUSIC RESOURCE MANAGEMENT

GROUP MEMBER:

1. CHENRUI DING, 2252734
2. JIN ZHOU, 2251739
3. XINYI YANG, 2251657
4. XU CHEN, 2257453
5. XUEXIN ZHAN, 2251792
6. YIYI ZHU, 2252441
7. YOU LONG XU, 2252978
8. ZICHEN QIU, 2252705



Xi'an Jiaotong-Liverpool University
西交利物浦大學

1.1 Background and Requirements

In recent years, the digital music landscape has evolved, increasing the demand for user-friendly music management platforms. Our project aims to develop a comprehensive music resource management system that meets both end-user and administrative needs.



QQ音乐

让生活充满音乐



1.2 Aims of The Project

Key requirements:

1. User Authentication
2. Music Management
3. Search Functionality
4. Data Security
5. Administrative Tools
6. User Feedback



1.3 User Characteristics



User Profile I

End User

- Registration & Login: Quick access to the system.
- Music Management: Upload, categorize, and manage music.
- Search & Playback: Find and play music.
- Privacy: Secure personal data.



User Profile II

System Administrator

- User Management: Control accounts and block inappropriate content.
- Content Review: Ensure compliance with platform rules.
- Reporting: Generate system and activity reports.

End users require easy registration, music management, fast search, secure playback, and data protection, while administrators manage accounts, moderate content, handle violations, and track system activity.



1.4 Assumptions and Dependencies

We ensured high system performance under peak traffic by using Alibaba Cloud for scalability, Docker and Docker Compose for stable container management, Spring Boot with MySQL and MyBatis-Plus for efficient data processing, GitHub for code collaboration, Spring Security for user protection, and JLayer for seamless MP3 playback, delivering a stable, efficient, and user-friendly system.



1.5 Project Scope & 1.6 Project Risks

- profile management
- music resource management
- user feedback functions

Risks:

Requirements Dynamics Risk

Complexity Risk

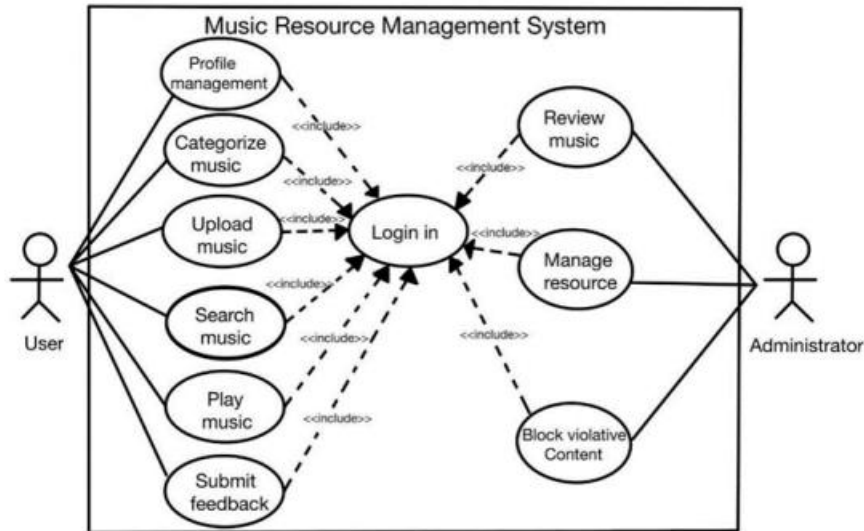
Security vulnerability Risk

Solution:

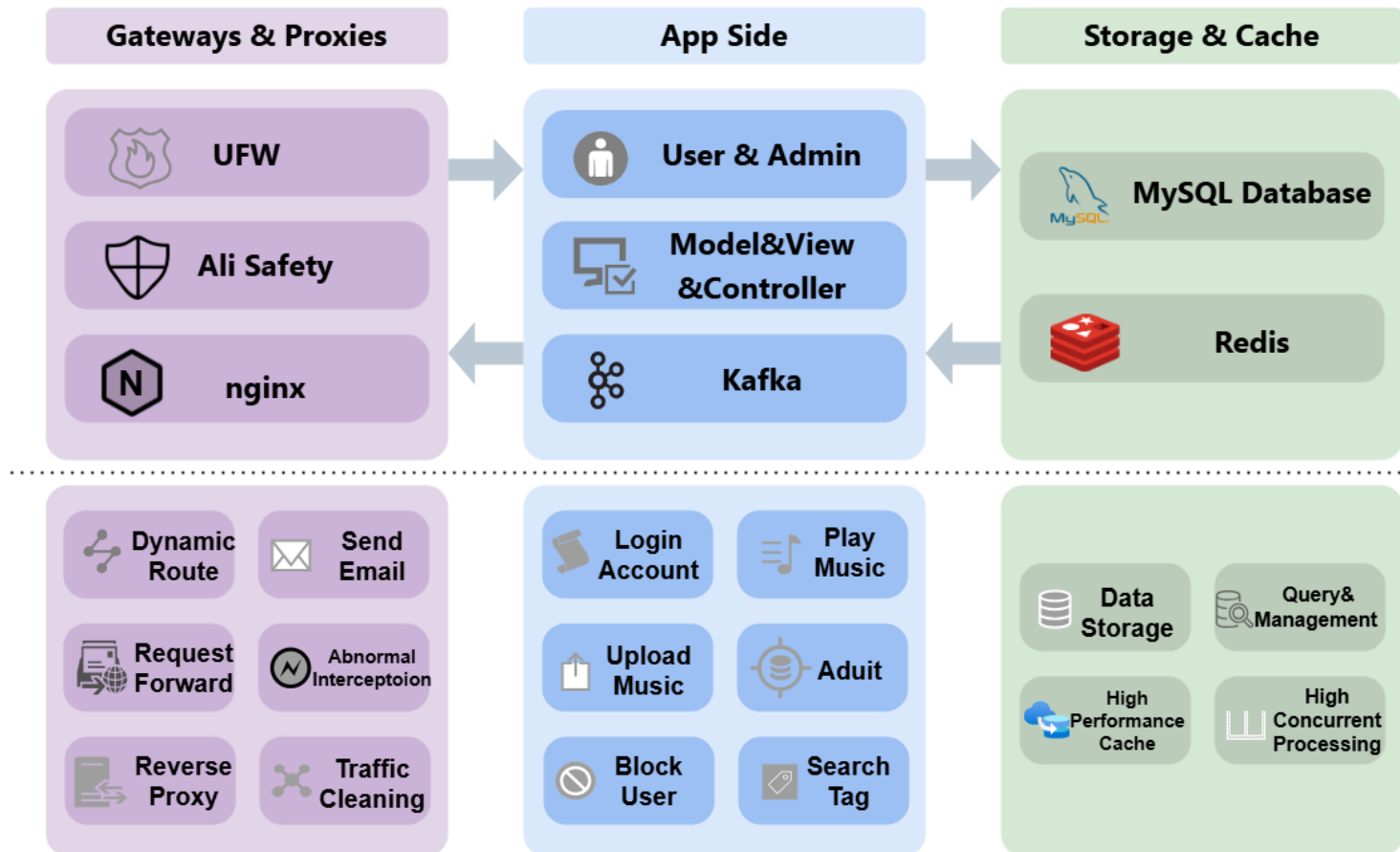
Session Management

Authentication

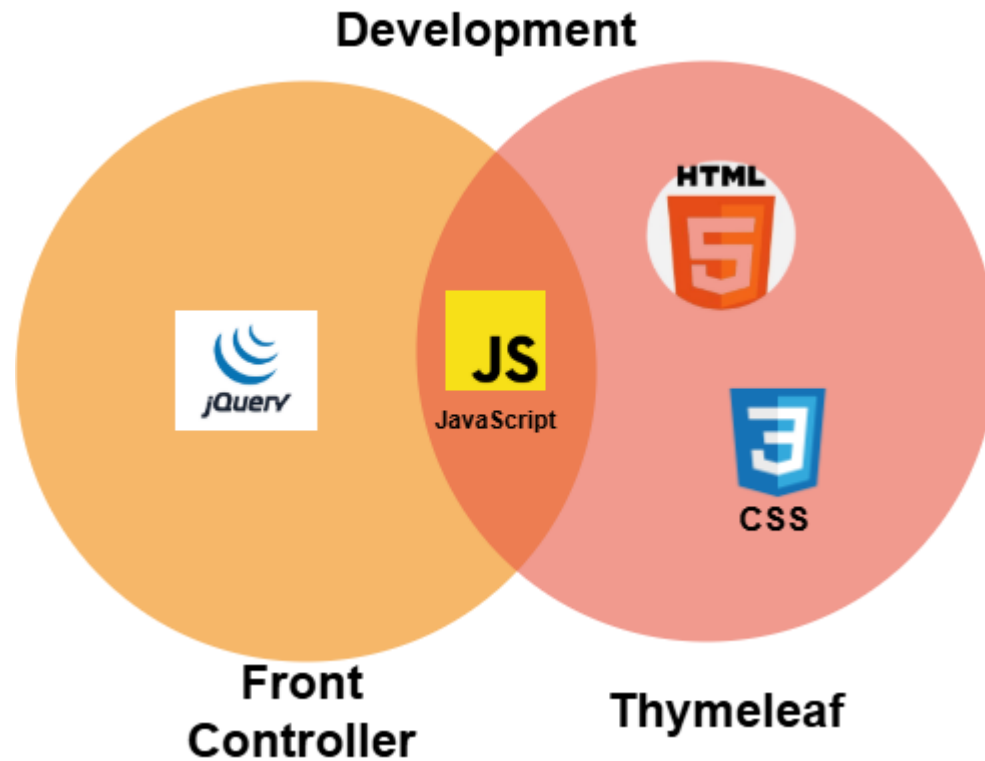
Data Encryption and Soft Deletion



2.1 System Architecture

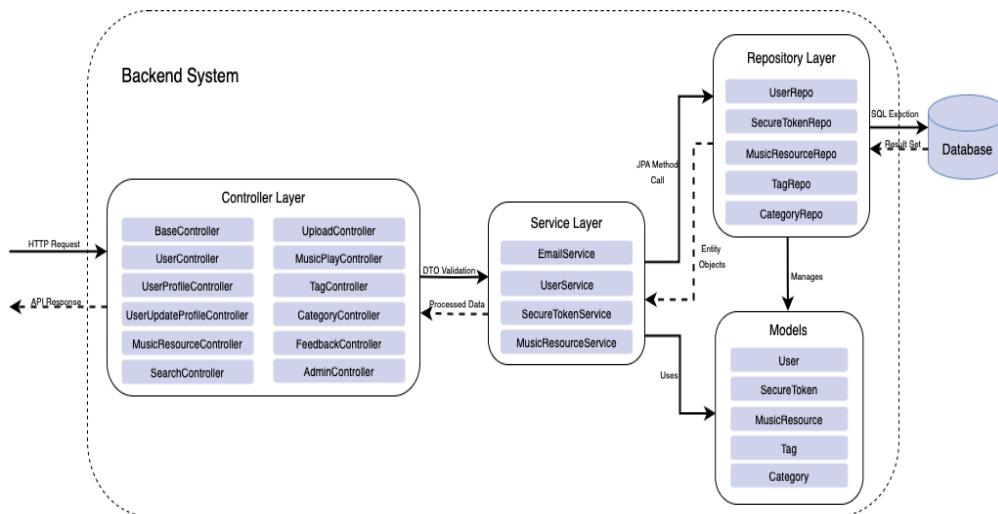


2.2 Frontend System



2.3 Backend System Architecture

Four-layer Architecture



Technology Stack

- **Multipart File interface:** Simplifies file upload operations, ensuring efficient handling of user avatar uploads and music submissions
- **JavaMail API:** Support email functionalities like user registration and password reset
- **JWT:** Implements token-based user authentication, ensuring system security and efficiency
- **Spring Data JPA:** Reduces boilerplate code for database operations, improving development efficiency



2.4 High-level Database Design

ER Diagram

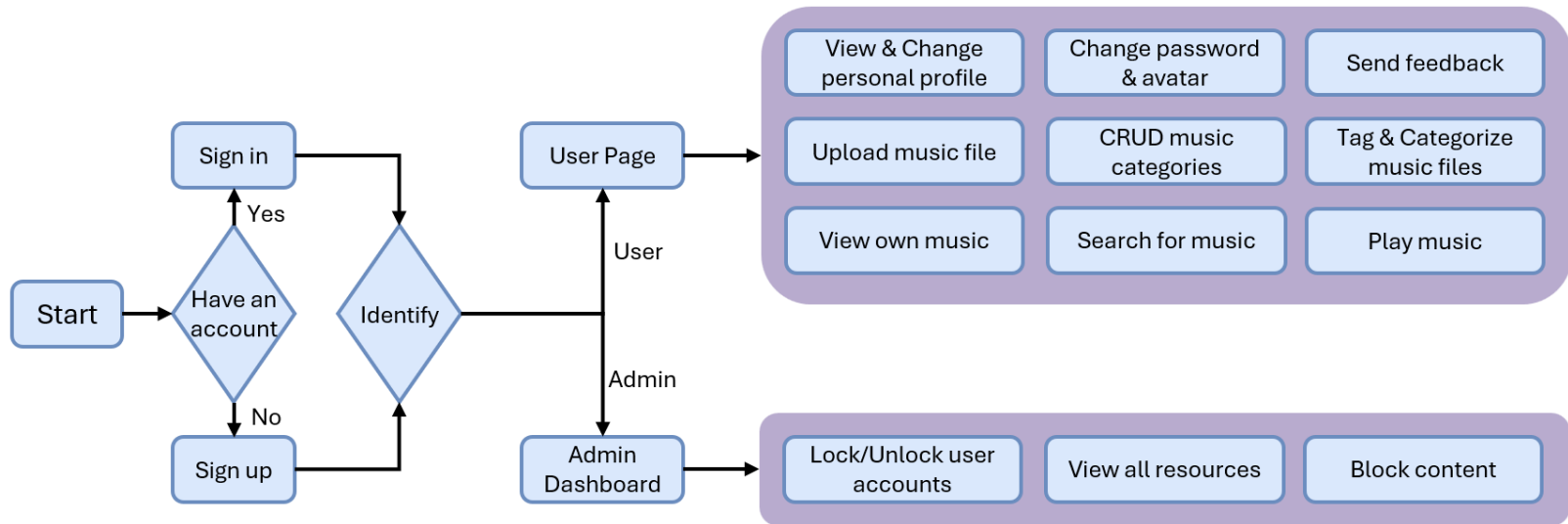


Design Principles

- **Normalization (3NF)**: Ensures data integrity and eliminates redundancy.
- **Foreign Key Constraints**: Maintains referential integrity.
- **Extensibility**: Supports dynamic system expansion.
- **User-Centric**: Ensures data isolation in a multi-user environment.



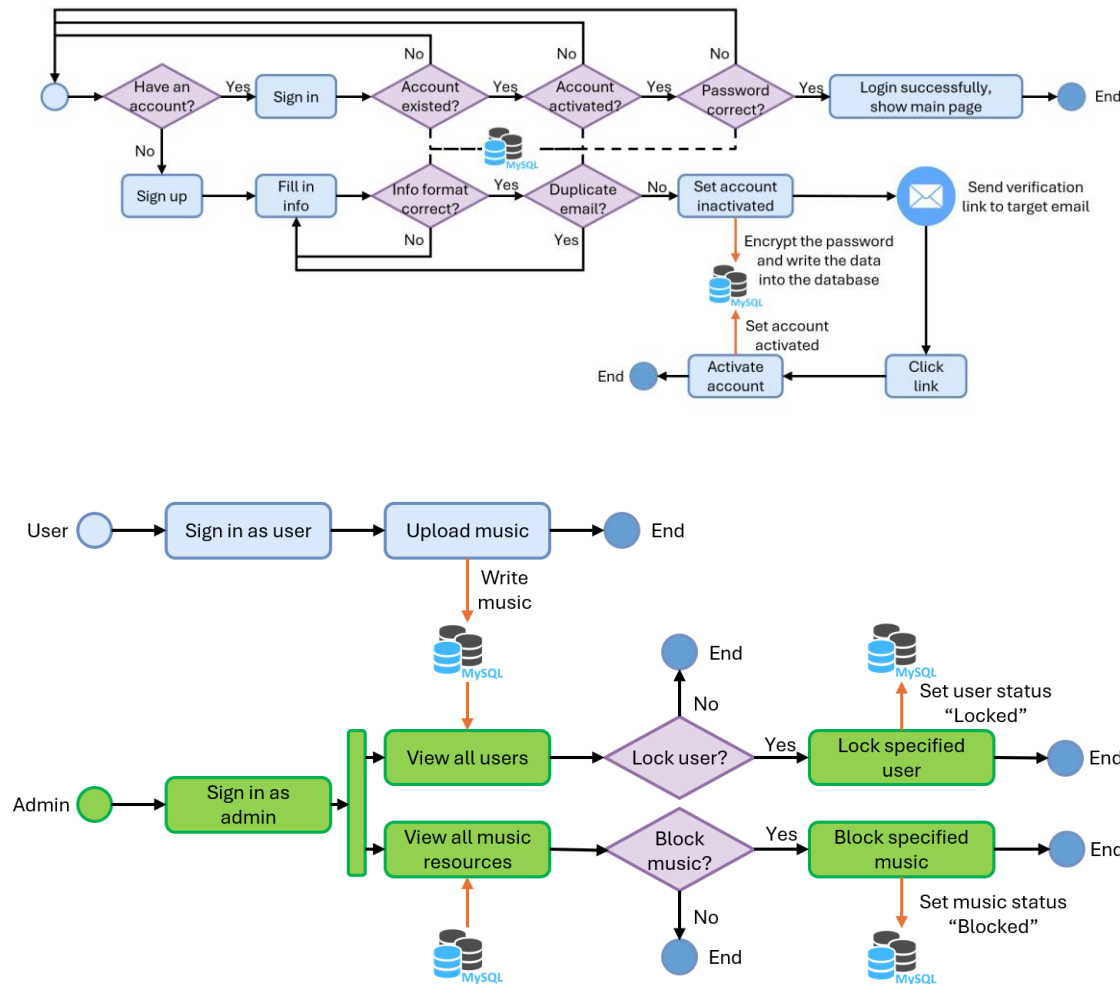
3.1 Software Modules



The Music Resource Management system uses role-based modules—**user** for music tools and **admin** for governance—supported by shared backend services, ensuring security and scalability. Strict RBAC isolates workflows: users upload and play content, while admins moderate, with no overlap. This modular design simplifies updates, prevents vulnerabilities, and adapts seamlessly to new roles or features.



3.2 High-level Process Flow

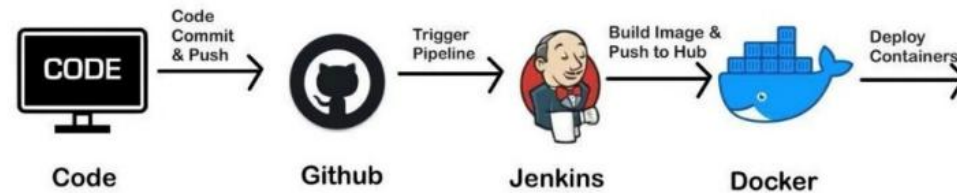


This platform uses separate workflows for users and administrators to ensure security and efficiency. Users enjoy fast, secure sign-up with instant verification, while admins oversee content, accounts, and audits via a specialized dashboard. Its role-based design with isolated systems prevents conflicts and ensures reliability.



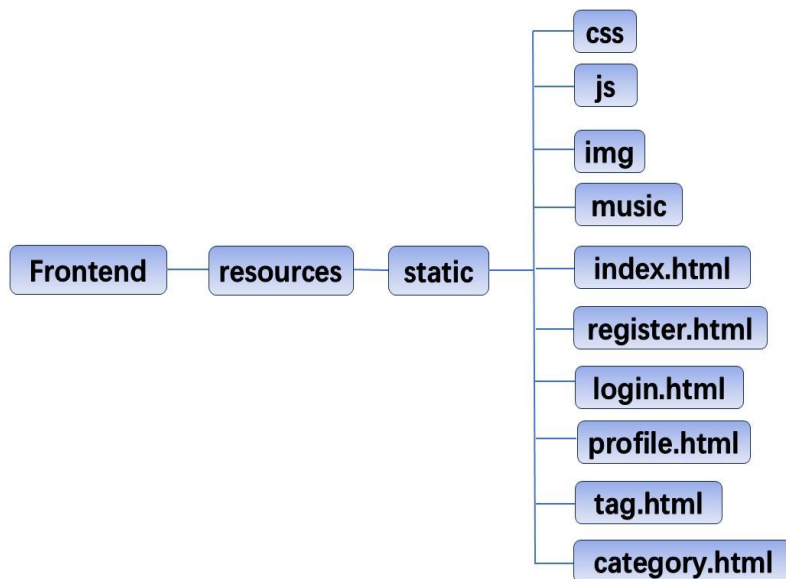
3.3 Software Support Services

- CI/CD pipeline automated deployment
- Nginx reverse proxy
- Docker containerized deployment
- MySQL
- Redis

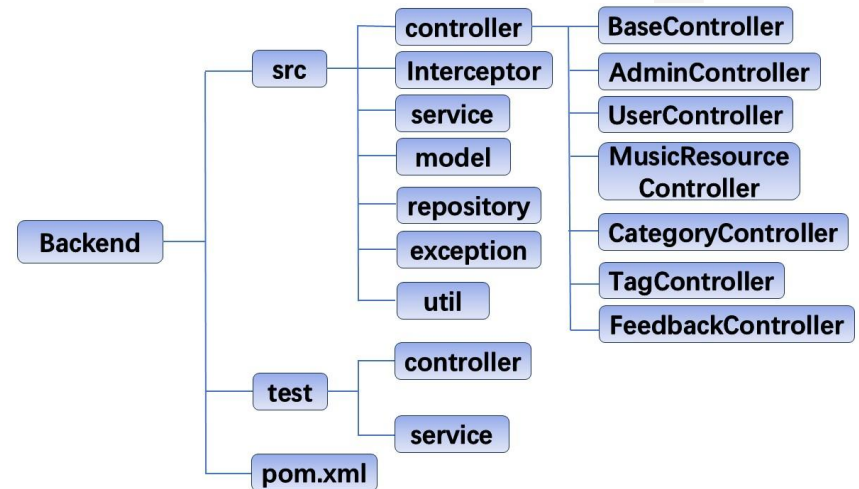


3.4 Coding Structure and Convention

Front-end part



Back-end part



3.5 Software Configuration and Production Environment

Table 1: Server Configuration

Field	Description
Operating System	Ubuntu 22.04
Core	4 Cores
RAM	8G
Bandwidth	5 Mbps
Upload Path	/cpt202/data
Max Upload Size	10MB
Max Request Size	20MB
JDK Version	JDK 17
Docker Version	26.1.3

Table 2: Software Configuration

Field	Version	IP	Port
Jenkins	2.492.3 LTS	127.0.0.1	8081
Nginx	1.27.5	172.30.0.4	80,443
Mysql	8.0.18	172.30.0.4	3306
Redis	6.2.12	172.30.0.5	6379
Springboot	3.4.3	172.30.0.6	8080

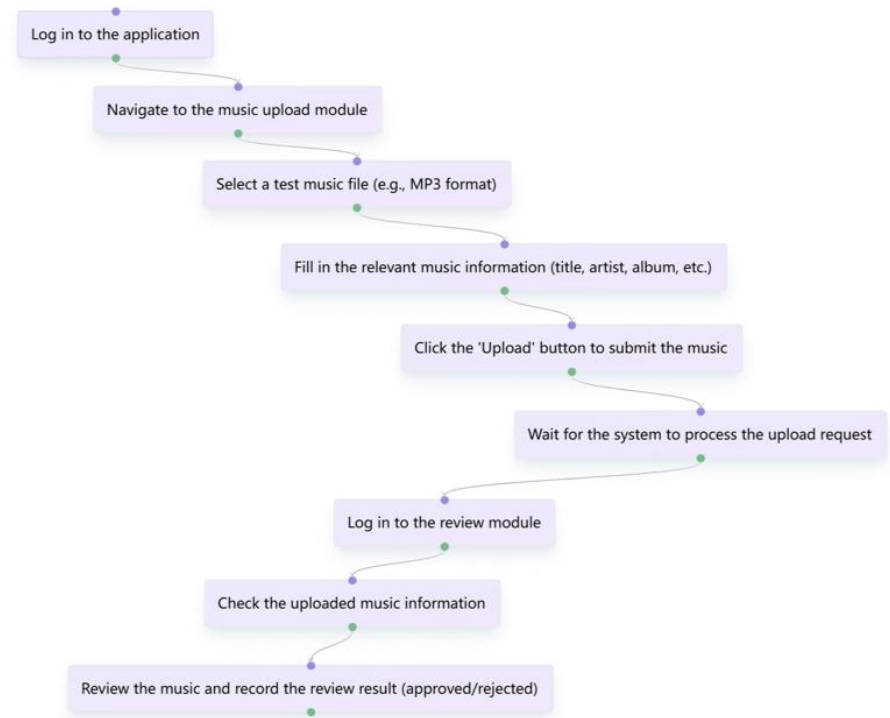
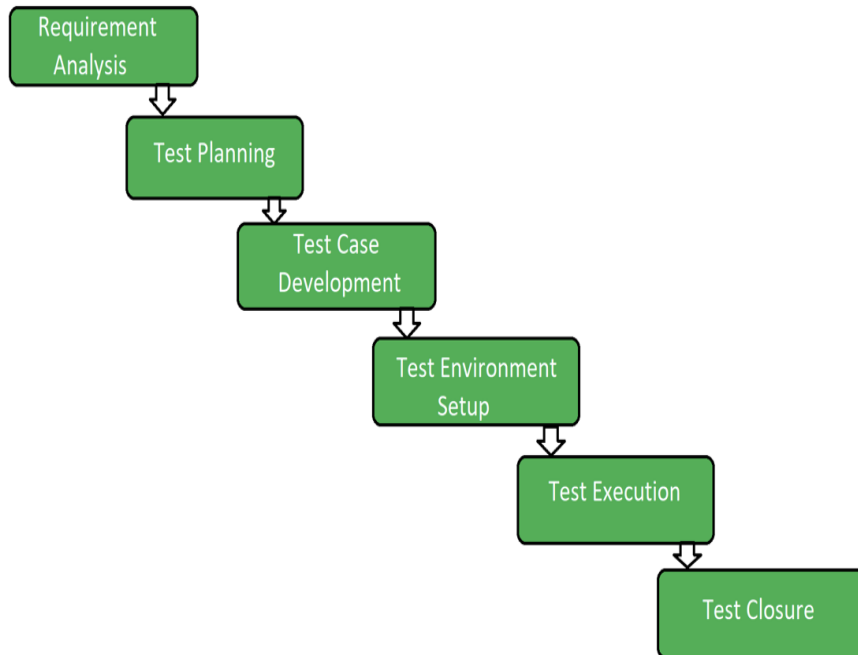


4.1 Unit Test

We used JUnit to test key functions like registration, login, upload, and playback, covering edge cases. Core features were validated, regression tests conducted, and important test code included in the appendix.

✓ UserUpdateProfileControllerTest (com.cpt202.group7.controller) 1sec 114 ms	✓ TagControllerTest (com.cpt202.group7.controller) 645 ms
✓ testSendVerificationEmail_Success() 1sec 66 ms	✓ testAddCategory() 634 ms
✓ testUpdateProfileFromUpdateController_Success() 19 ms	✓ testGetAllCategories() 6 ms
✓ testShowUpdateForm_Success() 10 ms	✓ testEditCategory() 3 ms
✓ testConfirmUpdate_Success() 8 ms	✓ testDeleteCategory() 2 ms
✓ testUpdatePwd_Success() 11 ms	
✓ UserControllerTest (com.cpt202.group7.controller) 1sec 334 ms	✓ MusicPlayControllerTest (com.cpt202.group7.controller) 867 ms
✓ testPostRegisterPage_Success() 1sec 58 ms	✓ testPlay_WithPartialParameters() 855 ms
✓ testPostLoginPage_Success() 228 ms	✓ testPlay_WithMissingParameters() 6 ms
✓ testVerifyUser_Failure() 10 ms	✓ testPlay_WithValidParameters() 6 ms
✓ testPostRegisterPage_EmailAlreadyRegistered() 8 ms	
✓ testGetRegisterPage() 6 ms	
✓ testGetAvatar() 16 ms	
✓ testVerifyUser_Success() 8 ms	

4.2 Integration Testing



4.3 Acceptance Testing

Acceptance testing confirmed the system met user needs, validating core functions, stability, and security. We initiated multi-language support and planned further improvements.

Functionality:

Core features tested (registration, login, upload, playback).

User Experience:

Intuitive interface, fast feedback, basic multi-language.

Performance:

Stable under high load, database optimized.

Security:

Basic protection done, further enhancements planned.

Compatibility:

Cross-device/browser support, English & Chinese available.



5 Conclusion and Future Work

- What make our music management web app outstanding?
 1. Protecting users' personal privacy (Registration & Email Verification)
 2. Enabling personalized music uploads (Tag-based categorization)
 3. Resource reviewing and user feedback.

- In what ways could our product be further developed in the future?
 - A. Improving interface explainability will optimize its user-friendly.
 - B. Applying digital integration strategies can expand commercial viability.

